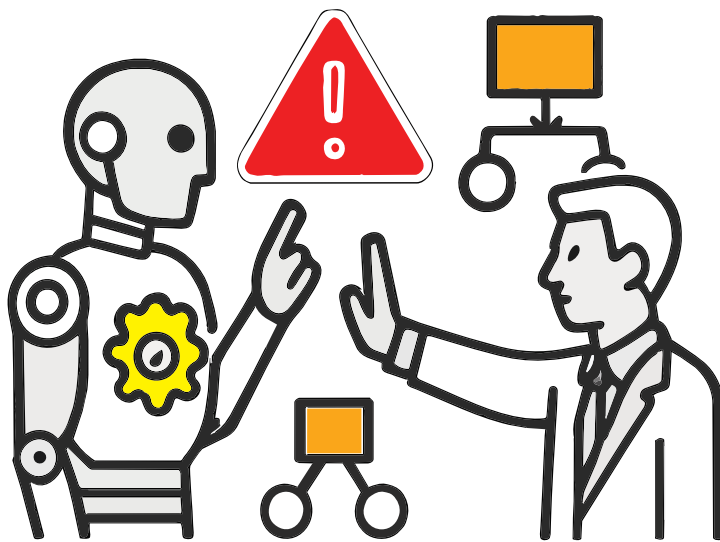


Stopping Dangerous AI Agent Actions Before Execution

A Linux-first control pattern can stop risky agent actions before they reach the shell or filesystem. You can enforce a policy before AI agent tool execution using open source and Linux security controls.



Put policy at the action boundary

At that boundary, normalise the proposed action into a small, deterministic record. For a shell tool, that can include the executable, arguments, working directory, environment references, and whether the command can reach the network. For a filesystem tool, capture the requested path and operation. For Git, capture whether the action is a read, commit, push, force-push, or history rewrite.

A policy check can then return one of three simple decisions: allow, deny, or require review. The important part is that the decision happens before the tool receives permission to execute.

```
action = {
  "tool": "shell",
  "command": "git push --force origin
main",
  "cwd": "/srv/project"
}
decision = policy.evaluate(action)
if decision == "deny": stop()
if decision == "review": request_
approval()
if decision == "allow": execute()
```

This pattern keeps the security decision outside the model. The agent can propose an action, but it cannot grant itself permission. That separation is the same reason Linux administrators do not treat an application process as the authority for its own access controls.

Use Linux controls as backstops

Pre-execution agent policy should

Imagine a coding agent working inside a Linux project. It can inspect files, run tests, edit source code, install packages, and call network tools. Most of those actions are ordinary. The dangerous moment is the one where a bad instruction, compromised dependency, or poisoned context turns an ordinary task into a command that reads a secret, deletes the wrong directory, or sends data somewhere it should not go.

If the security control only notices the problem after the command has run, the log may be useful for an investigation, but it is too late to prevent the action. A safer design puts a policy decision directly in front of the tool call.

The dangerous gap is before execution

AI agents do not need root access

to cause meaningful damage. A developer account can usually modify a repository, read configuration files, run a package manager, invoke Git, and reach the network. An agent that inherits those permissions inherits the same blast radius. The model may be the component deciding what to do, but the shell, filesystem, package manager, browser, and API clients are the components that actually do it.

That distinction matters. Monitoring model output is helpful, but a model can describe one action and a tool adapter can execute another representation. Post-execution telemetry is also helpful, but it records the event after the side effect. The reliable control point is the boundary where a proposed action has been converted into a concrete tool call but has not yet executed.